12.3 核心方法与事件

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200,000 tokens，输出 ~15,000 tokens（本章合计）

前两节我们理解了协议的设计哲学和类型系统。现在来看具体的：客户端通过这套协议能做什么？服务器会推送什么事件？本节逐一分析 Gateway 提供的核心方法和事件。

12.3.1 请求方法一览

Gateway 支持的所有请求方法定义在 `src/gateway/server-methods-list.ts` 中：

```ts
// src/gateway/server-methods-list.ts
const BASE_METHODS = [
  // ── 系统与运维 ──
  "health",               // 健康检查
  "status",               // 系统状态
  "logs.tail",            // 实时日志

  // ── 通道管理 ──
  "channels.status",      // 通道连接状态
  "channels.logout",      // 登出通道

  // ── 配置操作 ──
  "config.get",           // 获取配置
  "config.set",           // 设置配置
  "config.apply",         // 应用配置变更
  "config.patch",         // 部分更新配置
  "config.schema",        // 获取配置 Schema

  // ── 执行审批 ──
  "exec.approvals.get",  // 获取审批策略
  "exec.approvals.set",  // 设置审批策略
  "exec.approval.request",  // 请求审批
  "exec.approval.resolve",  // 批准/拒绝

  // ── 引导向导 ──
  "wizard.start",         // 启动引导向导
  "wizard.next",          // 下一步
  "wizard.cancel",        // 取消向导
  "wizard.status",        // 向导状态

  // ── AI 对话 ──
  "agent",                // 发起 Agent 对话
  "agent.identity.get",  // 获取 Agent 身份信息
  "agent.wait",           // 等待 Agent 运行完成
  "send",                 // 发送消息到通道
  "wake",                 // 唤醒 Agent

  // ── 模型与技能 ──
  "models.list",          // 列出可用模型
  "agents.list",          // 列出 Agent 配置
  "agents.files.list",    // 列出 Agent 文件
  "agents.files.get",     // 读取 Agent 文件
  "agents.files.set",     // 写入 Agent 文件
  "skills.status",        // 技能状态
  "skills.bins",          // 技能二进制
  "skills.install",       // 安装技能
  "skills.update",        // 更新技能
```

```
// ── 会话管理 ──
"sessions.list",        // 列出会话
"sessions.preview",     // 预览会话内容
"sessions.patch",       // 修改会话属性
"sessions.reset",       // 重置会话
"sessions.delete",      // 删除会话
"sessions.compact",     // 压缩会话

// ── 节点操作 ──
"node.pair.request",    // 请求节点配对
"node.pair.list",       // 列出配对请求
"node.pair.approve",    // 批准配对
"node.pair.reject",     // 拒绝配对
"node.pair.verify",     // 验证配对
"node.rename",          // 重命名节点
"node.list",            // 列出节点
"node.describe",        // 描述节点
"node.invoke",          // 调用节点
"node.invoke.result",   // 返回节点调用结果
"node.event",           // 节点事件

// ── 设备管理 ──
"device.pair.list",     // 列出设备配对请求
"device.pair.approve",  // 批准设备配对
"device.pair.reject",   // 拒绝设备配对
"device.token.rotate",  // 轮换设备令牌
"device.token.revoke",  // 撤销设备令牌

// ── 定时任务 ──
"cron.list",            // 列出定时任务
"cron.status",          // 任务状态
"cron.add",             // 添加任务
"cron.update",          // 更新任务
"cron.remove",          // 删除任务
"cron.run",             // 手动运行任务
"cron.runs",            // 运行历史

// ── 心跳与在线状态 ──
"last-heartbeat",       // 最近一次心跳
"set-heartbeats",       // 设置心跳
"system-presence",      // 系统在线信息
"system-event",         // 系统事件

// ── TTS（文字转语音） ──
"tts.status",           // TTS 状态
"tts.providers",        // TTS 供应商列表
"tts.enable",           // 启用 TTS
```

```
  "tts.disable",          // 禁用 TTS
  "tts.convert",          // 文字转语音
  "tts.setProvider",      // 设置供应商

  // ── WebChat 方法 ──
  "chat.history",         // 聊天历史
  "chat.send",            // 发送聊天消息
  "chat.abort",           // 中止聊天

  // ── 其他 ──
  "usage.status",         // 使用量统计
  "usage.cost",           // 费用统计
  "update.run",           // 运行更新
  "talk.mode",            // 对话模式切换
  "voicewake.get",        // 语音唤醒设置
  "voicewake.set",        // 语音唤醒设置
  "browser.request",      // 浏览器操作请求
];
```

方法列表并不是静态的。 `listGatewayMethods()` 函数会动态合并通道插件注册的额外方法：

```
export function listGatewayMethods(): string[] {
  const channelMethods = listChannelPlugins()
    .flatMap((plugin) => plugin.gatewayMethods ?? []);
  return Array.from(new Set([...BASE_METHODS, ...channelMethods]));
}
```

> **衍生解释**：这种**插件扩展**的设计模式在 Web 框架中非常常见。例如，Express.js 的中间件可以注册新的路由，Webpack 的插件可以注册新的生命周期钩子。OpenClaw 的通道插件也可以向 Gateway 注册新的方法——比如 Slack 扩展可能会注册 `slack.interactivity` 方法来处理 Slack 的交互式消息回调。

方法使用**点号分隔命名空间**（Dot-separated Namespace），这是一种清晰的组织方式：

命名空间	含义	示例
`config.*`	配置操作	`config.get` , `config.set`
`sessions.*`	会话管理	`sessions.list` , `sessions.delete`
`node.*`	节点操作	`node.invoke` , `node.list`
`device.*`	设备管理	`device.pair.approve`
`cron.*`	定时任务	`cron.add` , `cron.remove`
`skills.*`	技能管理	`skills.install` , `skills.status`
`exec.*`	执行审批	`exec.approval.request`

命名空间	含义	示例
`chat.*`	WebChat	`chat.send` , `chat.history`

这种设计对于连接握手时的**功能协商**（Feature Negotiation）也很关键——`HelloOk` 响应中的 `features.methods` 数组告诉客户端当前 Gateway 支持哪些方法。

12.3.2 `connect` — 握手与认证

`connect` 是客户端与 Gateway 建立通信的第一步。严格来说， `connect` 不是一个标准的请求方法（它在 WebSocket 连接建立后立即由客户端发送，而非通过 `req` 帧调用），但它遵循类似的请求-响应模式。

连接请求（ConnectParams）

```
// src/gateway/protocol/schema/frames.ts
export const ConnectParamsSchema = Type.Object({
  // 协议版本协商
  minProtocol: Type.Integer({ minimum: 1 }),  // 客户端支持的最低协议版本
  maxProtocol: Type.Integer({ minimum: 1 }),  // 客户端支持的最高协议版本

  // 客户端身份信息
  client: Type.Object({
    id: GatewayClientIdSchema,          // 客户端类型（如 "cli", "openclaw-macos"）
    displayName: Type.Optional(NonEmptyString),
    version: NonEmptyString,             // 客户端版本号
    platform: NonEmptyString,            // 运行平台（如 "darwin", "win32"）
    deviceFamily: Type.Optional(NonEmptyString),   // 设备系列
    modelIdentifier: Type.Optional(NonEmptyString),// 硬件型号
    mode: GatewayClientModeSchema,       // 客户端模式（如 "cli", "ui"）
    instanceId: Type.Optional(NonEmptyString),     // 实例 ID
  }),

  // 能力声明
  caps: Type.Optional(Type.Array(NonEmptyString)),
  commands: Type.Optional(Type.Array(NonEmptyString)),
  permissions: Type.Optional(Type.Record(NonEmptyString, Type.Boolean())),

  // 认证信息
  auth: Type.Optional(Type.Object({
    token: Type.Optional(Type.String()),    // Token 认证
    password: Type.Optional(Type.String()), // 密码认证
  })),

  // 设备认证（非对称密钥签名）
  device: Type.Optional(Type.Object({
    id: NonEmptyString,
    publicKey: NonEmptyString,
    signature: NonEmptyString,     // 签名
    signedAt: Type.Integer(),      // 签名时间戳
    nonce: Type.Optional(NonEmptyString), // 随机数（v2）
  })),

  // 其他
  role: Type.Optional(NonEmptyString),
  scopes: Type.Optional(Type.Array(NonEmptyString)),
  locale: Type.Optional(Type.String()),
  userAgent: Type.Optional(Type.String()),
  pathEnv: Type.Optional(Type.String()),
});
```

这个结构承载了大量信息。分组解析：

协议版本协商： `minProtocol` 和 `maxProtocol` 让客户端和服务器能协商使用一个双方都支持的协议版本。当前协议版本是 3。如果服务器不支持客户端要求的版本范围，连接会被拒绝。

客户端身份： `client` 字段标识了"谁在连接"。这不仅用于日志记录，还影响 Gateway 的行为——例如， `mode: "cli"` 的客户端和 `mode: "ui"` 的客户端可能有不同的权限和功能。

能力声明： `caps` （Capabilities）让客户端声明自己支持的额外能力。目前定义的能力包括：

```
export const GATEWAY_CLIENT_CAPS = {
  TOOL_EVENTS: "tool-events",  // 客户端支持接收工具事件
} as const;
```

连接响应（HelloOk）

连接成功后，Gateway 返回 `HelloOk` 消息：

```typescript
export const HelloOkSchema = Type.Object({
  type: Type.Literal("hello-ok"),
  // 协商结果
  protocol: Type.Integer({ minimum: 1 }),  // 双方同意的协议版本

  // 服务器信息
  server: Type.Object({
    version: NonEmptyString,    // Gateway 版本号
    commit: Type.Optional(NonEmptyString),   // Git commit
    host: Type.Optional(NonEmptyString),     // 主机名
    connId: NonEmptyString,     // 连接 ID（唯一标识此次连接）
  }),

  // 功能清单
  features: Type.Object({
    methods: Type.Array(NonEmptyString),  // 支持的方法列表
    events: Type.Array(NonEmptyString),   // 支持的事件列表
  }),

  // 当前状态快照
  snapshot: SnapshotSchema,

  // 连接策略
  policy: Type.Object({
    maxPayload: Type.Integer({ minimum: 1 }),      // 最大消息大小（字节）
    maxBufferedBytes: Type.Integer({ minimum: 1 }), // 最大缓冲大小
    tickIntervalMs: Type.Integer({ minimum: 1 }),   // 心跳间隔（毫秒）
  }),

  // 认证结果
  auth: Type.Optional(Type.Object({
    deviceToken: NonEmptyString,  // 分配的设备令牌
    role: NonEmptyString,         // 分配的角色
    scopes: Type.Array(NonEmptyString), // 权限范围
    issuedAtMs: Type.Optional(Type.Integer()),
  })),

  canvasHostUrl: Type.Optional(NonEmptyString),
});
```

几个关键点：

1. **`features.methods`** 和 **`features.events`** 告诉客户端当前 Gateway 支持哪些操作。这实现了**运行时功能发现**——客户端不需要硬编码一个方法列表，而是在连接时动态获取。
2. **`snapshot`** 包含了连接时刻的系统状态（在线用户、健康状况等），客户端可以用它初始化 UI，而不需要再发额外的请求。

3. **`policy`** 告知客户端连接的限制参数。 `maxPayload` 防止客户端发送过大的消息， `tickIntervalMs` 告诉客户端心跳频率。

12.3.3 `agent` / `agent.wait` — 发起 AI 对话

`agent` 方法是 OpenClaw 最核心的操作——它让客户端发起一次 AI 对话。

agent 方法参数

```
// src/gateway/protocol/schema/agent.ts
export const AgentParamsSchema = Type.Object({
  // 必填参数
  message: NonEmptyString,           // 用户消息
  idempotencyKey: NonEmptyString,    // 幂等键

  // Agent 选择
  agentId: Type.Optional(NonEmptyString),  // 指定 Agent（留空使用默认）

  // 会话选择
  sessionId: Type.Optional(Type.String()),   // 会话 ID
  sessionKey: Type.Optional(Type.String()),  // 会话键
  label: Type.Optional(SessionLabelString),  // 会话标签

  // 消息路由
  to: Type.Optional(Type.String()),          // 发送目标
  replyTo: Type.Optional(Type.String()),     // 回复目标
  channel: Type.Optional(Type.String()),     // 通道
  replyChannel: Type.Optional(Type.String()),// 回复通道
  accountId: Type.Optional(Type.String()),   // 账户 ID
  replyAccountId: Type.Optional(Type.String()),
  threadId: Type.Optional(Type.String()),    // 线程 ID（如 Slack 线程）
  groupId: Type.Optional(Type.String()),     // 群组 ID
  groupChannel: Type.Optional(Type.String()),
  groupSpace: Type.Optional(Type.String()),

  // 行为控制
  thinking: Type.Optional(Type.String()),     // 思考级别
  deliver: Type.Optional(Type.Boolean()),     // 是否投递回复
  timeout: Type.Optional(Type.Integer()),     // 超时（毫秒）
  lane: Type.Optional(Type.String()),         // 执行车道（并发控制）
  extraSystemPrompt: Type.Optional(Type.String()), // 额外系统提示词

  // 附件
  attachments: Type.Optional(Type.Array(Type.Unknown())),

  // 子 Agent 追踪
  spawnedBy: Type.Optional(Type.String()),
});
```

这是整个协议中最复杂的参数结构之一。字段多的原因是 `agent` 方法需要同时处理来自多种通道的消息——CLI 发来的消息可能只需要 `message` 和 `idempotencyKey`，而来自 Slack 群组的消息还需要 `threadId`、`groupId`、`channel` 等路由信息。

agent.wait — 等待运行完成

`agent` 方法是**异步的**——它立即返回一个 `runId`，实际的 Agent 运行在后台进行，结果通过事件推送。但有些场景需要**同步等待**结果，比如 CLI 执行 `openclaw ask "什么是 OpenClaw?"`。

`agent.wait` 方法满足这个需求：

```
export const AgentWaitParamsSchema = Type.Object({
  runId: NonEmptyString,                          // 要等待的运行 ID
  timeoutMs: Type.Optional(Type.Integer({ minimum: 0 })), // 等待超时
});
```

调用流程：

```
Client ── req(method="agent", params={message:"Hello"}) ──▶ Server
       ◀── res(payload={runId:"run_abc"}) ──────────────── Server
       ── req(method="agent.wait", params={runId:"run_abc"}) ──▶ Server
       ◀── event(agent, payload={stream:"text", data:...}) ── Server
       ◀── event(agent, payload={stream:"text", data:...}) ── Server
       ◀── res(payload={...final result...}) ──────────────── Server
```

12.3.4 `send` — 发送消息到通道

`send` 方法允许 Agent 或客户端向特定的通道发送消息：

```
export const SendParamsSchema = Type.Object({
  to: NonEmptyString,              // 发送目标（用户ID、群组ID等）
  message: NonEmptyString,         // 消息内容
  mediaUrl: Type.Optional(Type.String()),   // 单个媒体 URL
  mediaUrls: Type.Optional(Type.Array(Type.String())), // 多个媒体 URL
  gifPlayback: Type.Optional(Type.Boolean()),  // GIF 播放标记
  channel: Type.Optional(Type.String()),       // 指定通道
  accountId: Type.Optional(Type.String()),     // 指定账户
  sessionKey: Type.Optional(Type.String()),    // 关联会话
  idempotencyKey: NonEmptyString,  // 幂等键
});
```

`send` 和 `agent` 的区别在于：`agent` 是"请 AI 处理这条消息"，而 `send` 是"直接发送这条消息到目标通道"。`send` 通常由 Agent 在工具调用中使用（比如 Agent 决定向某个 Slack 频道发送一条通知）。

`PollParamsSchema` 是 `send` 的变体，用于发送投票：

```
export const PollParamsSchema = Type.Object({
  to: NonEmptyString,
  question: NonEmptyString,
  options: Type.Array(NonEmptyString, { minItems: 2, maxItems: 12 }),
  maxSelections: Type.Optional(Type.Integer({ minimum: 1, maximum: 12 })),
  durationHours: Type.Optional(Type.Integer({ minimum: 1 })),
  channel: Type.Optional(Type.String()),
  accountId: Type.Optional(Type.String()),
  idempotencyKey: NonEmptyString,
});
```

投票选项限制在 2–12 个之间，最大可选择数量限制为 12——这些都是通过 TypeBox Schema 的约束直接表达的。

12.3.5 `sessions.*` — 会话管理方法族

会话是 OpenClaw 中 AI 对话的上下文容器。`sessions.*` 方法族提供了会话的完整 CRUD 操作：

方法	功能	参数特点
`sessions.list`	列出会话	支持过滤：`limit`、`activeMinutes`、`label`、`agentId`、`search`
`sessions.preview`	预览会话内容	支持多会话批量预览，可限制字符数
`sessions.resolve`	解析会话	通过 `key`、`sessionId`、`label` 等多种方式定位会话
`sessions.patch`	修改会话属性	可修改 `label`、`model`、`thinkingLevel`、`execSecurity` 等
`sessions.reset`	重置会话	清空对话历史，保留会话键
`sessions.delete`	删除会话	可选是否同时删除转录文件
`sessions.compact`	压缩会话	调用 LLM 总结旧消息，减少上下文长度

`sessions.patch` 的参数特别丰富，反映了会话的可配置性：

```
export const SessionsPatchParamsSchema = Type.Object({
  key: NonEmptyString,
  label: Type.Optional(Type.Union([SessionLabelString, Type.Null()])),
  thinkingLevel: Type.Optional(Type.Union([NonEmptyString, Type.Null()])),
  reasoningLevel: Type.Optional(Type.Union([NonEmptyString, Type.Null()])),
  model: Type.Optional(Type.Union([NonEmptyString, Type.Null()])),
  execHost: Type.Optional(Type.Union([NonEmptyString, Type.Null()])),
  execSecurity: Type.Optional(Type.Union([NonEmptyString, Type.Null()])),
  sendPolicy: Type.Optional(Type.Union([
    Type.Literal("allow"),
    Type.Literal("deny"),
    Type.Null(),
  ])),
  // ... 更多可配置字段
});
```

注意一个设计模式：几乎所有字段都是 `Type.Union([SomeType, Type.Null()])` ——传值时设置，传 `null` 时重置为默认值，不传时保持不变。这是一种优雅的**三态更新**（Three-State Update）模式。

12.3.6 事件类型

除了请求方法，Gateway 还会主动推送事件。所有事件类型定义在 `GATEWAY_EVENTS` 数组中：

```
// src/gateway/server-methods-list.ts
export const GATEWAY_EVENTS = [
  "connect.challenge",      // 连接挑战（设备认证）
  "agent",                  // Agent 运行事件（流式输出）
  "chat",                   // WebChat 消息事件
  "presence",               // 在线状态变更
  "tick",                   // 心跳
  "talk.mode",              // 对话模式变更
  "shutdown",               // 服务器关闭通知
  "health",                 // 健康状态变更
  "heartbeat",              // 心跳事件
  "cron",                   // 定时任务触发
  "node.pair.requested",    // 节点配对请求
  "node.pair.resolved",     // 节点配对结果
  "node.invoke.request",    // 节点调用请求
  "device.pair.requested",  // 设备配对请求
  "device.pair.resolved",   // 设备配对结果
  "voicewake.changed",      // 语音唤醒设置变更
  "exec.approval.requested",// 执行审批请求
  "exec.approval.resolved", // 执行审批结果
];
```

agent 事件

`agent` 事件是最核心的事件类型，它承载了 Agent 运行过程中的所有流式输出：

```
export const AgentEventSchema = Type.Object({
  runId: NonEmptyString,    // 运行 ID（关联到哪次 agent 调用）
  seq: Type.Integer({ minimum: 0 }),  // 序列号（保证事件顺序）
  stream: NonEmptyString,   // 流名称（如 "text"、"tool-call"、"tool-result"）
  ts: Type.Integer({ minimum: 0 }),   // 时间戳
  data: Type.Record(Type.String(), Type.Unknown()), // 事件数据
});
```

`stream` 字段区分了不同类型的事件数据：

- `"text"` — 模型生成的文本块
- `"tool-call"` — 模型决定调用某个工具
- `"tool-result"` — 工具执行结果
- `"thinking"` — 模型的思考过程
- `"error"` — 运行时错误

`seq` 序列号保证即使事件到达顺序被打乱（在网络层面极少发生，但在多路复用场景中理论上可能），客户端也能正确重排。

tick 事件

`tick` 是一个简单的心跳事件：

```
export const TickEventSchema = Type.Object({
  ts: Type.Integer({ minimum: 0 }),  // 服务器时间戳
});
```

Gateway 按 `policy.tickIntervalMs` 指定的间隔发送 `tick` 事件。客户端据此：

1. 确认连接仍然活跃
2. 同步与服务器的时钟差

shutdown 事件

当 Gateway 准备关闭时，会发送 `shutdown` 事件给所有连接的客户端：

```
export const ShutdownEventSchema = Type.Object({
  reason: NonEmptyString,  // 关闭原因（如 "upgrade"、"manual"）
  restartExpectedMs: Type.Optional(Type.Integer({ minimum: 0 })),
  // 预计重启时间（如果有的话）
});
```

`restartExpectedMs` 是一个贴心的设计——如果 Gateway 因为升级而重启，客户端知道大约多久后可以尝试重连，而不需要盲目重试。

状态版本与增量同步

事件帧可以携带一个 `stateVersion` 字段：

```
export const StateVersionSchema = Type.Object({
  presence: Type.Integer({ minimum: 0 }),  // 在线状态版本
  health: Type.Integer({ minimum: 0 }),    // 健康状态版本
});
```

> **衍生解释**：状态版本（State Version）是一种**乐观并发控制**（Optimistic Concurrency Control）技术。每当某个状态发生变化时，版本号递增。客户端记住当前的版本号，如果收到的事件的版本号大于自己记录的版本号，就知道状态发生了变化需要更新。这比每次都获取完整状态要高效得多。
>
> 类似的设计在数据库中也很常见——例如 PostgreSQL 的 MVCC（多版本并发控制）和 etcd 的 revision 机制。

本节小结

Gateway 的方法和事件设计体现了几个原则：

1. **功能分族**：通过点号命名空间将 80+ 个方法组织为清晰的功能组
2. **异步优先**：`agent` 方法是异步的，结果通过事件流推送；`agent.wait` 提供同步等待的选择
3. **丰富的元数据**：`connect` 握手时交换大量信息（版本、能力、状态快照），减少后续的往返通信
4. **三态更新模式**：`sessions.patch` 等方法使用"传值设置、传 null 重置、不传保持"的模式
5. **事件驱动**：18 种事件类型覆盖了系统状态变更的方方面面，客户端可以根据需要订阅感兴趣的事件

在下一节中，我们将深入 Gateway 的认证机制——Token 认证、设备配对和 Origin 检查是如何保护 WebSocket 连接安全的。